



中山大學
SUN YAT-SEN UNIVERSITY

实验六：Pthreads 并行构造

并行程序设计与算法实验报告

2026-05-18

元朗曦 23336294

计算机学院 计算机科学与技术

计八

一 实验目的

本实验围绕基于 Pthreads 的并行循环构造展开，目标是将较底层的线程创建、任务分解与同步过程封装为可复用的 `parallel_for` 接口，并进一步使用该接口改写规则网格上的热传导程序。通过本实验，希望达到以下目的：

1. 理解共享内存环境下并行循环的分解、分配与执行过程。
2. 使用 Pthreads 实现类似 OpenMP `parallel for` 的抽象，并生成动态链接库。
3. 以矩阵乘法验证 `parallel_for` 的正确性与可复用性。
4. 将 `heated_plate_openmp` 改写为基于 Pthreads 的并程序，并比较不同线程数、不同调度方式下的程序表现。

二 实验原理

2.1 Pthreads 并行循环构造

`parallel_for` 的基本思想是将一个逻辑循环区间拆分给多个线程执行。实验中实现的基础接口为：

```
int parallel_for(int start, int end, int inc,
                void *(*functor)(int, void *),
                void *arg, int num_threads);
```

其中，`start`、`end`、`inc` 描述循环索引范围，`functor` 表示每次迭代真正执行的函数，`arg` 用于传入共享参数，`num_threads` 表示期望创建的线程数。线程并不关心具体业务逻辑，只负责领取迭代并调用 `functor`，因此调度机制与计算内容可以彼此分离。

若逻辑迭代总数为 n ，线程数为 p ，则静态调度会在循环开始前把任务近似均匀切成 p 份。该方式调度开销较小，适合每次迭代工作量接近的循环。动态调度则使用原子计数器保存“下一段待执行任务”，线程完成当前任务后继续领取新的任务块，负载均衡能力更强，但需要额外同步。

2.2 矩阵乘法

设 $A \in \mathbb{R}^{m \times n}$ 、 $B \in \mathbb{R}^{n \times k}$ ，则矩阵乘法结果 $C \in \mathbb{R}^{m \times k}$ 满足：

$$C_{i,j} = \sum_{p=0}^{n-1} A_{i,p} B_{p,j}$$

矩阵 C 的每一行都可独立计算，因此可将外层行循环交给 `parallel_for` 分配。不同线程只写入各自负责的行，不会产生写冲突。对方阵规模 $n \times n$ 的测试，浮点运算量约为 $2n^3$ ，可用运行时间与串行版本比较加速效果。

2.3 Heated plate 问题

heated plate 用离散迭代模拟平板上的稳态热传导。对于内部网格点，其下一轮温度由上下左右四个邻居的平均值给出：

$$w_{i,j}^{t+1} = \frac{1}{4} (u_{i-1,j}^t + u_{i+1,j}^t + u_{i,j-1}^t + u_{i,j+1}^t)$$

每轮迭代主要包含三步：

1. 将当前温度矩阵 `w` 复制到旧矩阵 `u`；
2. 根据 `u` 更新内部点的温度；
3. 计算本轮前后温度变化的最大值 `diff`，当 `diff ≤ epsilon` 时停止。

由于每一行的复制、更新和局部误差计算都可以独立完成，本实验将这些行级循环封装为多个 `functor`，再统一交给 `parallel_for` 执行。

三 实验内容

3.1 `parallel_for` 动态库实现

实验在 `lab/06/src` 中实现 `parallel_for.h` 与 `parallel_for.c`，并通过 `Makefile` 生成动态链接库 `libparallel_for.so`。除实验要求中的基础接口外，程序还提供了扩展接口：

```
int parallel_for_schedule(int start, int end, int inc,
                        void *(*functor)(int, void *),
                        void *arg, int num_threads,
                        pf_schedule_t schedule, int chunk_size);
```

其中 `schedule` 可选 `PF_STATIC` 与 `PF_DYNAMIC`。静态调度按线程数一次性划分迭代区间；动态调度借助 `atomic_int next_iteration` 分配任务块。程序同时支持正向与反向循环，并对空循环、非法步长和非法线程数做了基本检查。

3.2 矩阵乘法验证程序

`matmul_test.c` 使用 `parallel_for_schedule` 按行计算矩阵乘法，并在同一次运行中计算串行结果，随后比较最大绝对误差。典型调用方式如下：

```
./matmul_test 256 4 static
./matmul_test 256 4 dynamic
```

该程序能够同时验证动态库接口是否正确，以及在规则负载下不同调度方式的开销差异。

3.3 Heated plate 的 Pthreads 改造

`heated_plate_pthreads.c` 将原 OpenMP 版本中的多个并行循环改写为基于 `parallel_for` 的调用，主要包括：

1. 边界初始化；
2. 内部网格初始化；
3. 每轮迭代中的矩阵复制、温度更新和逐行最大误差计算。

与原程序相比，新程序保留了相同的物理模型与停止条件，并将线程数、调度方式、网格规模和误差阈值设计为命令行参数：

```
./heated_plate_pthreads [threads] [static|dynamic] [m] [n] [epsilon]
```

为了便于和原始 OpenMP 版本比较, 实验还提供 `benchmark.sh`, 可自动运行 Pthreads 的静态/动态调度版本以及原始 OpenMP 程序。

3.4 构建与运行

```
cd lab/06/src
make
./matmul_test 128 4 static
./heated_plate_pthreads 4 dynamic 64 64 0.01
```

`Makefile` 会同时生成 `libparallel_for.so`、`matmul_test`、`heated_plate_pthreads` 与用于对照的 `heated_plate_openmp`。

四 实验结果

4.1 正确性验证

在矩阵规模为 128×128 、线程数为 4 时, 静态调度版本输出如下:

```
matrix_size=128 threads=4 schedule=static
parallel_time=0.000880 s serial_time=0.000508 s speedup=0.577
max_error=0 result=PASS
```

最大绝对误差为 0, 说明 `parallel_for` 对矩阵乘法的任务划分与执行结果正确。由于该规模较小, 线程创建和同步开销相对计算量更显著, 因此并行版本并未获得明显加速; 这也说明并行化并非在任意问题规模上都天然更快。

在 64×64 网格、4 线程、动态调度、误差阈值为 0.01 的 heated plate 测试中, 程序在第 762 次迭代后收敛:

```
HEATED_PLATE_PTHREADS
grid=64 x 64 threads=4 schedule=dynamic epsilon=0.01 mean=74.603175
...
    762  0.009980
Error tolerance achieved.
Wallclock time = 0.373654
```

收敛过程与原算法预期一致, 说明 Pthreads 改造版本能够正确完成热传导迭代。

4.2 调度方式比较

在相同的 64×64 heated plate 测试中, 静态与动态调度的结果如下:

调度方式	线程数	迭代次数	运行时间 / s
static	4	762	0.430575
dynamic	4	762	0.373654

表 1 不同调度方式下 heated plate 的测试结果

两种调度方式得到完全一致的迭代次数与最终误差，说明调度策略不影响数值正确性。该测试中动态调度略快，但由于每次迭代的行计算量本身较均匀，二者差距并不构成绝对结论；在更大网格和多次重复实验下，静态调度通常更容易体现低开销优势，而动态调度更适合工作量不均的任务。

4.3 与 OpenMP 的性能对比

为比较 Pthreads 构造与原始 OpenMP 程序的并行性能，使用默认的 500×500 网格与误差阈值 $\varepsilon = 0.001$ 进行测试。结果如下：

实现	线程数	调度方式	运行时间 / s	相对加速比
Pthreads	1	static	11.185514	1.000
Pthreads	2	static	11.193122	0.999
Pthreads	4	static	11.890744	0.941
Pthreads	8	static	17.730994	0.631
OpenMP	1	default	8.150603	1.000
OpenMP	2	default	4.062493	2.006
OpenMP	4	default	2.176018	3.746
OpenMP	8	default	1.674335	4.868

表 2 heated plate 在不同线程数下的性能对比

从结果可见，OpenMP 版本随线程数增加呈现出明显加速，而当前 Pthreads 版本并未体现出同样的扩展性，甚至在线程数较多时出现退化。其根本原因并非计算结果错误，而是当前 `parallel_for` 每次调用都会重新创建并回收线程；heated plate 在每轮迭代中又要连续执行多次并行循环，因此线程管理开销被反复放大。OpenMP 运行时通常会复用线程团队，因此在这类多轮短循环任务上更具优势。

4.4 结果分析

本实验表明，`parallel_for` 可以把 Pthreads 中繁琐的线程管理收束为更接近 OpenMP 的调用形式，使业务代码只需关注“每次迭代做什么”。对于矩阵乘法、heated plate 这类天然按行拆分的问题，这种抽象能够较自然地复用。

同时，实验也显示出并程序的真实边界：线程数增加并不必然带来线性收益，问题规模、线程创建成本、调度策略和同步次数都会共同决定最终性能。尤其在 heated plate 中，每轮迭代都要执行多个并行循环，若线程被反复创建和回收，管理开销会被持续放大。后续若继续优化，可进一步考虑线程池、持久工作线程和更细致的 chunk 调整，从而让这个并行构造更接近成熟运行时系统。